

prompt-evolution-ga

A Brief: what it does, what it found, and what to say about it

Explainer for Salman Adnan's portfolio

1 The idea

LLMs that write code are unreasonably sensitive to how you word the instructions. Finding the wording that works is normally a human guessing and re-running. This project asks whether a genetic algorithm can do that guessing, and, more importantly, builds a way to *prove* whether it worked.

The project in one sentence

A genetic algorithm whose population is English instruction prompts, whose fitness is “does the C++ this prompt produced actually compile and pass the real test cases”, and whose crossover and mutation operators are a second, stronger LLM.

Three choices define it:

1. **The chromosome is prose.** Not a bit-vector: an entire natural-language system prompt. This breaks the usual GA guarantees, and that turns out to matter.
2. **Fitness is objective.** A prompt is not judged by asking a model “is this good?”. It is judged by compiling the C++ it produced with `g++ -O2 -std=c++23`, running the binary against the problem's real test cases, and scoring the fraction that pass. You cannot argue with a failed test.
3. **The operators are an LLM.** You cannot splice two English paragraphs and get English, so Gemini 2.5 Pro is asked to merge, extend, trim or rephrase the prompt text.

Built for a Computational Intelligence course (paper co-authored with Ghufuran Alvi; the implementation is Salman Adnan's). Roughly 2,300 lines of Python across ten files, no framework, no database.

The headline, up front

The result is negative. With \$17 and 13,002 model calls across three runs, no evolved prompt beat the best hand-written seed on the 200-problem held-out test set (evolved 0.7857 vs baseline 0.7900). The project's value is that it is *capable of proving its own hypothesis wrong*, which most prompt-optimisation work is not, because most of it has no objective fitness function and no held-out test set. Defend the method; report the result.

2 How it works

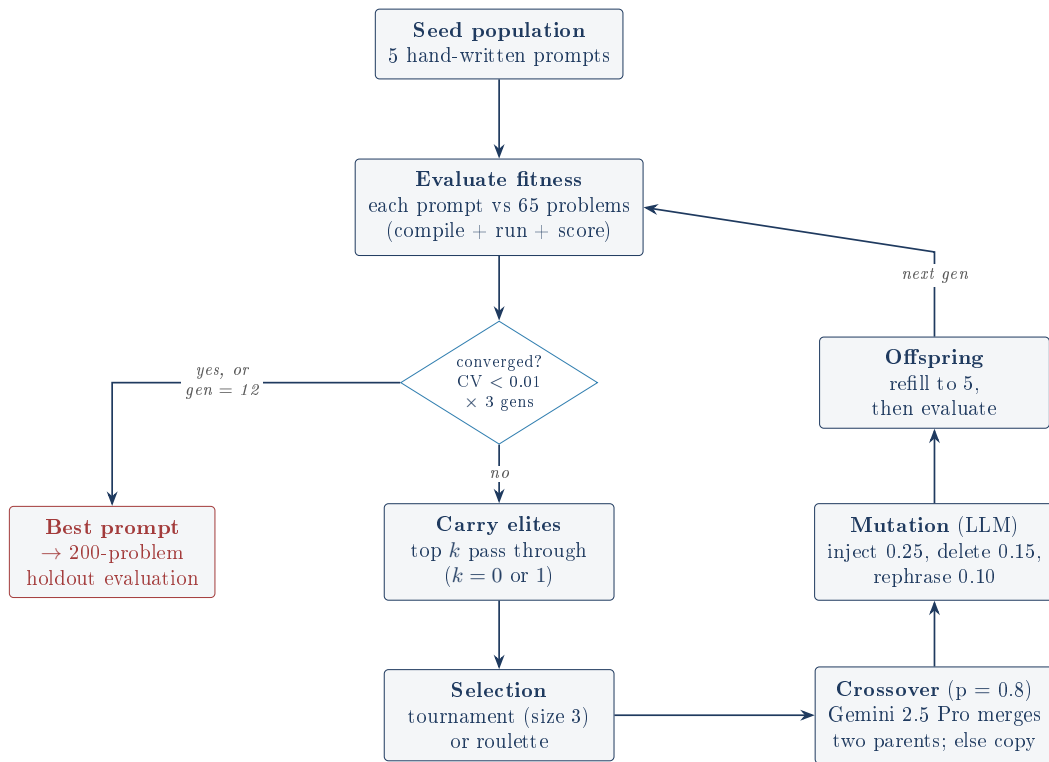


Figure 1: The generational loop: population 5, 12 generations, fitness measured on 65 problems, winner tested once on 200 unseen problems.

2.1 One fitness measurement

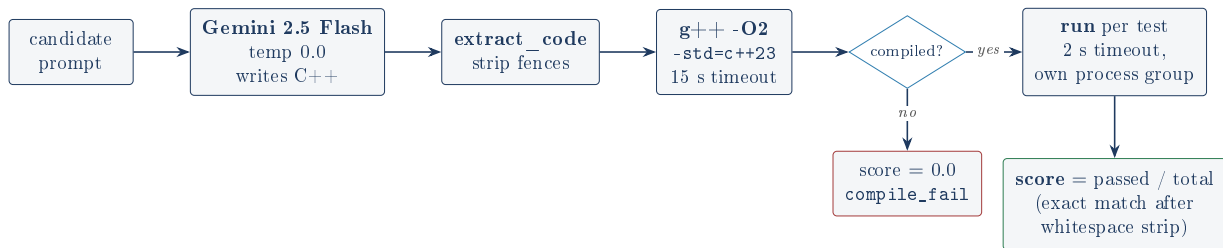


Figure 2: Repeated for all 65 problems, five at a time in a thread pool; the mean is the individual's fitness. Early stopping after 3 consecutive failures caps the cost.

2.2 The parts

File	Role
<code>ga.py</code>	The GA core: Individual , the two selection functions, breed , parallel fitness, the generational loop, RNG-state serialisation.
<code>main.py</code>	Orchestrator: pre-flight checks, split, checkpoints, resume, hold-out evaluation.
<code>llm.py</code>	The only file that talks to Vertex AI. Client construction, retry with exponential backoff and jitter, token and cost capture, the four operator prompts.
<code>evaluator.py</code>	Compile, run, score. Process-group kills, timeouts, output truncation.
<code>dataset.py</code>	Loads 300 hardcoded APPS problems; stratified split into 65 evolution / 200 holdout.
<code>cost_tracker.py</code>	Per-call token and dollar accounting, live JSON reports.
<code>seeds.py</code>	15 hand-written seed prompts (only the first 5 are used).
<code>config.py</code>	Every hyperparameter as a module constant.
<code>run_baseline_eval.py</code>	Evolved vs baseline on the 200-problem test split.
<code>visualize_test_results.py</code>	Rebuilds the 5 comparison figures, offline.

2.3 The engineering worth pointing at

- **Failure-conditioned mutation.** The `inject` operator is fed the parents' actual failures (compile errors, expected-vs-actual output) and asked to add a strategy targeting them. That is a directed mutation in a gradient-free method, and it is the most original idea here.
- **RNG state checkpointing.** The full Mersenne Twister state (625 elements) is serialised to JSON, so `--resume` continues with the exact coin flips the uninterrupted run would have made. Most projects seed a generator and call it reproducible.
- **Process-group kills.** Every subprocess is launched under `os.setsid` and killed with `os.killpg`, so a runaway program cannot leave orphans. `g++` itself forks, so this is not theoretical.
- **Thinking tokens costed correctly.** Gemini 2.5's hidden reasoning tokens are billed at the output rate and are folded in before costing, which is why the \$17 figure matches the bill.
- **The split is stored, not just seeded.** The chosen problem IDs are written out to `evolution_ids.json`, so a resumed run restores the exact split from disk, rather than trusting a seed to regenerate it.

3 What the results actually say

All numbers below were re-derived from the committed artifacts, not taken from the README.

Prompt	Mean pass rate (200 test problems)	vs baseline
Baseline (best hand-written seed)	0.7900	
Tournament evolved	0.7857	−0.0043
Roulette evolved	0.7717	−0.0183
Elitism evolved	0.7713	−0.0187

Table 1: Verified by recomputing from the four evaluation logs and by re-running `visualize_test_results.py`. No evolved prompt beat the hand-written baseline.

Per-problem, tournament vs baseline: **26 wins, 22 losses, 152 ties**. Three quarters of the

problems were unaffected by the prompt at all. Cost: 13,002 calls, \$17.05 (\$5.99 + \$6.16 + \$4.90), read from `Costs/`.

3.1 The finding the write-up misses

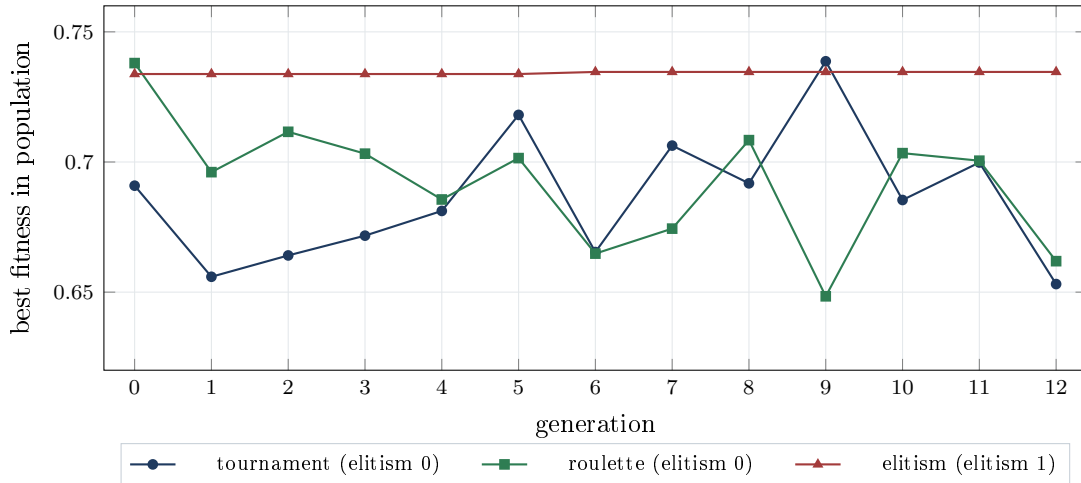


Figure 3: Best fitness per generation, from each run’s committed `history.json`. The two elitism-free runs wander and both end *below* where they started.

“Evolution reliably improved fitness” is not what the data shows

The README reports “best-of-run fitness 0.65 to 0.73”. Those are the *final* generation’s best. Against generation 0, the hand-written seeds:

run	gen 0 best	gen 12 best	change
tournament	0.6909	0.6531	−0.0378 (worse)
roulette	0.7380	0.6619	−0.0761 (worse)
elitism	0.7338	0.7346	+0.0008 (flat)

Two of three runs ended with a worse prompt than the seeds they started from. The third improved by noise. The tournament run peaked at 0.7387 at generation 9 and then lost that individual entirely.

The mechanism is textbook and easy to explain: with `elitism = 0` the whole population is replaced each generation, so nothing protects the best individual. A good prompt found at generation 9 is gone by generation 10 unless it is selected *and* survives an LLM rewriting it. With a population of 5, it usually does not. The one run that kept an elite is the one run whose curve never drops.

The paper’s claim that “the GA consistently produced prompts that outperformed the best” seed on the training subset is contradicted by the repository’s own data. **This is the single most important thing to be ready to discuss.**

Every winning prompt is a truncated fragment

`results/run_20260429_210411/best_prompt.txt`, in full (160 characters):

```
1 First, approach the problem systematically: analyze the input-output relationship to
2 identify the core algorithm, paying close attention to variable types, edge
```

It stops mid-sentence. So do the other two winners (120 and 132 characters, ending “...first

perform a” and “...include main(),”). All three are far shorter than the roughly 500-character seeds they descend from.

Cause: `API_FALLBACK_MAX_TOKENS_OPS = 700` caps operator output, and `breed`’s guard only checks `res.strip()`, which is truthy for any non-blank text including a mid-word truncation. The API’s `finish_reason` field would reveal it and is never read.

This reframes the headline. The comparison is not “evolved vs hand-written”; it is “a truncated 160-character fragment vs a complete hand-written prompt”, and the fragment lost by 0.0043. That a fragment came that close is arguably interesting, but it is not the experiment as described. The README treats this as a one-run artifact; it affects all three.

4 Where the documentation and the code disagree

The license contradiction: fix this first

README line 79 says “Code is MIT licensed (see LICENSE).” The `LICENSE` file is **PolyForm Strict 1.0.0**, which permits viewing only and forbids reuse and redistribution. These are close to opposites. Commit `38c7f8b` switched the license and never updated the README. It is the only defect here with legal meaning, and it is a one-line fix.

The g++ failure mode is described backwards

The README claims the fitness path “fails with a plain ‘g++ not found’ rather than silently scoring zero”. Verified on a machine with no g++: it returns `score = 0.0`, `reason = compile_fail`, and the GA carries on. A run without g++ would burn 12 generations and \$6, score every individual exactly 0.0, and report a random winner. `RUNNING.md` describes this correctly; the README describes the opposite. The fix is a `shutil.which("g++")` line in `main.py`’s pre-flight, which already checks the credential and the dataset but not the compiler.

The paper’s abstract claims comparisons that were never run

The abstract says evolved prompts “outperform hand-crafted baselines, chain-of-thought prompts, and random search”. Verified by grep: **neither a chain-of-thought comparison nor a random-search baseline exists anywhere in the codebase**. And the one comparison that does exist went the wrong way. The word “preliminary” does not carry this much weight.

The genealogy feature: broken, and its script is not in the repo

The README headlines “full genealogy logging” and the abstract claims genealogy analysis “reveals which instructional strategies survive selection pressure”. The data capture is real (parents, operators and intermediate prompt states are written to every checkpoint), but the committed analysis found nothing: `genealogy_tree.json` for the flagship run reads `{"uid": "efd4a762", "note": "not found in checkpoints"}` and the report says “Lineage depth: 0 steps”.

The root cause is a one-line omission: `save_checkpoint` only ever fires at the *top* of the loop, so the **final generation is never written to disk**. Verified: all three runs have 12 checkpoints but 13 generations of history, and the winners of the tournament and roulette runs appear in *no* checkpoint. The elitism run escapes only because its winner was an elite carried forward from generation 6.

Separately, **no Python file in the repository contains the string “genealogy”**. The reports are committed outputs of a script that was never committed, so the headline analysis

feature cannot be re-run or fixed by anyone reading the repo.
The same omission means `--resume` after a completed run silently rewinds to generation 11.

The recorded `elitism_count` is wrong in all three runs

`main.py` writes `"elitism_count": config.ELITISM_COUNT` (the module constant) rather than the value actually passed to `ga.run`. So the *elitism* run's own config file records elitism 0, when it used 1. The tournament run's records 2.

That 2 cannot have been in force: an elite mathematically guarantees best fitness never decreases, and that run's fitness decreases repeatedly. **Inferred, not verified**, and the artifacts do not record enough to settle it: all three runs are marked `resumed: true`, so a code change between the original run and its resume is possible.

Related: the committed run directories are bare timestamps, but `main.py` now creates `run_<timestamp>_<scheme>` and `RUNNING.md` documents that name. **The committed results were not produced by the committed code.** Every number in the repository comes from a version of the program that no longer exists in it.

The four-way comparison cannot be reproduced from the repo

`run_baseline_eval.py` hardcodes the tournament run directory and writes exactly two files. But `visualize_test_results.py` *requires* four and exits without them. The roulette and elitism logs were made by hand-editing constants, and that variant was never committed. Adding a `--scheme` flag is a fifteen-minute fix.

4.1 Smaller, but worth knowing

- **The holdout is easier than the training pool.** The evolution pool is drawn from `train/` (ratings 800 to 1900); the holdout only from `test/` (800 to 1400). So prompts tuned on problems up to rating 1900 are tested only up to 1400. Neither the README nor the paper mentions this.
- **The three “schemes” are two switches.** `elitism` is not a selection mechanism; it is tournament with one elite. Roulette-with-elitism was never run, so the design cannot separate “elitism helps” from “elitism helps tournament”.
- **Configured temperatures are dead.** `TARGET_TEMPERATURE` and `OPTIMIZER_TEMPERATURE` are defined in `config.py` and read *nowhere*; `llm.py` hardcodes the same values. Editing config would silently do nothing.
- `_pick_failures` has a **logic bug** (if reason not in seen or `len(picked) < 3`) that defeats its own intent: the first three failures picked are typically all the same reason, so the best idea in the project runs on a less varied sample than designed.
- `ops.append()` runs even when the operator returned **nothing**, so an individual's recorded operator string can claim a mutation that never happened.
- **Not a sandbox.** 13,000 pieces of model-written C++ were compiled and run on the host with no isolation. The 2-second timeout bounds how long arbitrary code runs, not what it can do.
- **Dead code:** `run_fitness_batch`, which only ever raises `NotImplementedError`; also `evaluator.run_tests`, `dataset.reserved_train_problems`, an unused `PRICING` import, and Claude pricing rows from a backend the project never used in its committed history.
- `corrections.tex` is a half-applied patch file for the paper, left at the repo root with no record of which parts landed.

- `main.py` prints “loading 305 pre-selected problems”; it loads 300.

5 Verification record

Executed while writing this, on a machine with Python 3.11+ and matplotlib but **no g++** and **no GCP credentials**:

- All 10 modules byte-compile. **Pass.**
- `dataset.load_problems()` loads **300** problems; `split` gives 65 / 200 / 35 exactly as configured.
- `visualize_test_results.py` regenerates all 5 figures and prints 0.7857 / 0.7717 / 0.7713 / 0.7900, matching the README exactly.
- Recomputing the means from the four JSONL logs independently reproduces the same four numbers. All logs: 200 entries, 0 infrastructure failures.
- Win/loss recomputed from `comparison.json`: 26 / 22 / 152. Matches.
- Cost totals from `Costs/`: 13,002 calls, \$17.05. Matches.
- `run_tests_verbose` without g++ returns score 0.0 / `compile_fail`. **Contradicts the README.**
- Best-fitness monotonicity from `history.json`: tournament and roulette non-monotonic and ending below gen 0; elitism monotonic.
- Winner present in a checkpoint: tournament **no**, roulette **no**, elitism yes.
- `grep`: no genealogy script, no chain-of-thought baseline, no random search, no reader of the temperature constants.

Not verified: the evolution loop, the Vertex AI calls, and the compile-and-run fitness path. These need a billed GCP project and g++. Every fitness number here is read from committed artifacts, not reproduced.

Labelled inferred: that the tournament run used elitism 0 despite its config recording 2; that the 700-token cap interacts with the 2048-token thinking budget to cause the truncation; that `seeds.py`’s ten unused prompts are leftovers from the paper’s population-15 setup; that the roulette and elitism comparison logs were made by hand-editing `run_baseline_eval.py`.

6 The scorecard

6.1 Genuinely good

The fitness function (compile-and-run is the right answer and the hard one; it is what makes the negative result trustworthy). Failure-conditioned mutation. RNG-state checkpointing. Process-group kills. Cost accounting that includes thinking tokens. Results that are auditable offline: the analysis pipeline reproduces its own headline numbers exactly, and I checked. And a README that already admits the headline negative result and several of its own bugs, which is rarer than it should be.

6.2 Fix first, in order of damage

1. **The license contradiction.** One line. The only defect with legal meaning.
2. **The “evolution reliably improved fitness” claim,** in both the README and the paper. Two of three runs regressed below their seeds.
3. **The paper’s abstract,** claiming chain-of-thought and random-search comparisons that do not exist.

4. **Checkpoint the final generation.** One line; it silently breaks resume and is the root cause of the broken genealogy.
5. **Validate operator output for completeness** (check `finish_reason`). All three winners are truncated.
6. **Add g++ to the pre-flight**, and fix the backwards README sentence.
7. **Record the elitism actually used**, not the module constant.
8. **Commit the genealogy script**, or drop the claim and its orphaned outputs.
9. **Parameterise `run_baseline_eval.py` by scheme.**
10. **Statistical testing.** A 0.0043 gap over 200 problems needs a paired test or a bootstrap before it means anything either way.

The line to take

Lead with the method, not the result. This project built an objective, expensive, honest measuring instrument for a question usually answered by vibes, and then reported that the answer was no. The instrument works.

Then get ahead of the two things a careful reader finds: the winning prompts are all truncated fragments, and two of three runs ended below their starting seeds because nothing protected the best individual. Both have clean, one-sentence explanations. Both are more interesting than the headline. Neither is in the write-up yet, and fixing that is the next job.